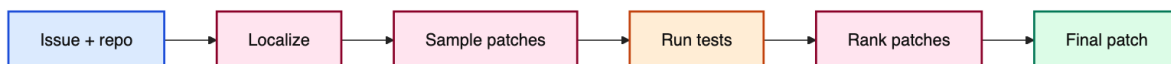


第 07 讲: Agentless: 为什么少一点自治有时更强

Coding Agent Notes (June 2026)

1 核心图景

Agentless 这个名字容易误导。它不是“不用 LLM”，也不是“不做软件工程 agent 该做的事情”。它真正去掉的是自由运行的在线决策 loop：不让模型在每一步自己决定下一步要不要查文件、运行命令、改代码、回头搜索，而是把 repo repair 拆成固定阶段，让模型在每个阶段只完成一个受约束的子任务。



这条 pipeline 可以读成一句话：先把 issue 和 repo 压缩成少量可能编辑的位置，再围绕这些位置采样多个小补丁，最后用 regression tests、生成的 reproduction test 和 majority voting 选择最终 patch。

Agentless 的中心主张：

当任务可以被拆成定位、修复、验证三个可审计阶段时，
减少在线自治反而可能提升稳定性、可解释性和成本控制。

这不是反对 autonomous agent。相反，Agentless 给了我们一个很好的系统设计边界：如果问题的关键不是“下一步该探索哪里”，而是“从有限候选中找到正确位置和正确 patch”，那么 batch pipeline 可能比自由交互 loop 更可靠。

1.1 一个端到端例子

假设 issue 说：某个日期解析函数在遇到空时区字符串时会抛出异常，期望行为是返回默认时区。一个自由 agent 可能会先搜索关键词，再打开多个文件，运行测试，临时修改代码，然后根据报错继续调整。Agentless 会把同一件事拆成更窄的步骤。

第一步，file localization 不需要理解完整实现，只要判断 issue 里的“timezone”、“parse”、“default”这些词可能对应哪些文件。它可能选出 `utils/datetime.py`、`tests/test_datetime.py` 以及一个配置文件。第二步，skeleton localization 只看这些文件里的函数和类名，例如 `parse_timezone`、`normalize_datetime`、`DateParser`，从中选择最相关元素。第三步，edit localization 才阅读 `parse_timezone` 的完整函数体，判断需要在空字符串分支加 default handling。

Repair 阶段不会再重新探索仓库，而是在这个局部窗口内生成多个 Search/Replace patches。有的 patch 可能只处理 None，有的 patch 处理空字符串，有的 patch 同时改测试。Validation 阶段再运行旧测试和生成的 reproduction test，把明显破坏旧行为的 patch 丢掉，并在剩余候选中投票。

这个例子说明了 Agentless 的核心：

它把“会不会修”拆成“能不能保留正确位置、能不能生成正确局部 diff、能不能选出正确候选”。

2 Agentless 不等于没有 LLM

Agentless 仍然大量使用 LLM：让模型根据 repo tree 和 issue 选择可疑文件，让模型阅读 skeleton 选择相关函数，让模型定位 edit locations，让模型生成 patches，让模型生成 reproduction tests，让模型判断哪些 passing tests 不应作为 regression tests。区别在于：模型不控制全局流程。

系统形态	谁决定下一步	状态形态
Autonomous loop	模型每轮选择 action	trajectory / observations
Agentless pipeline	系统固定 stage order	JSONL 中间产物
Hybrid system	系统给阶段，模型局部探索	stage state + trajectory

因此，Agentless 的“less”不是少模型能力，而是少 policy freedom。它把模型从一个在线操作者变成一组子程序的生成器：localization generator、patch generator、test generator、test selector。每次 LLM 调用都有明确输入、明确输出格式和明确下游用途。

2.1 中间产物像数据库表

Agentless repo 的 SWE-bench pipeline 会把每一阶段产物落成 JSONL 文件。这个工程细节很重要：它让系统可以暂停、复跑、替换某个阶段的模型、比较不同 localization 设置，而不必从头重跑整条 agent trajectory。

Artifact	保存什么	调试价值
loc_outputs.jsonl	file-level localization	看真实文件是否进入候选
combined_locs.jsonl	prompt + embedding 合并结果	分析召回来自哪一路
output.jsonl	sampled patches	对比不同 location set 的修复
test_results.jsonl	regression / reproduction signals	复查 patch selection 是否合理

这和 autonomous loop 的日志不同。trajectory log 记录的是“模型当时做了什么”，而 JSONL stage artifacts 记录的是“每个阶段产生了哪些候选对象”。前者适合复盘行为，后者适合做系统工程实验。

2.2 为什么这会更稳

自由 agent loop 的优点是灵活。模型可以根据 observation 改变策略，遇到意外环境问题可以探索，定位失败可以换路线。但灵活性也带来 decision entropy：每一步都可能走歪，每个错误 observation 都可能影响后续 trajectory，每次工具失败都可能消耗预算。

Agentless 把不确定性移到两个地方：**候选生成**和**候选选择**。它不要求一条轨迹从头到尾都正确，而是允许定位和修复阶段产生多个候选，再让 validation 去筛选。这种结构很像搜索系统：先扩大召回，再用更便宜、更可执行的信号排序。

Insight. Agentless 的关键不是“少做事”，而是把在线决策换成离线候选集。它牺牲了一部分交互灵活性，换来更强的 stage attribution 和批量验证。

3 Repo repair 为什么能拆成 search

一个 SWE-style issue repair 至少包含三个问题。

- **在哪里改**：相关文件、函数、类、行号是什么？
- **怎么改**：在这些位置生成什么 patch？
- **信哪个**：多个 patch 中哪个最可能真的修复 issue 且不破坏旧行为？

Agentless 的三个阶段正好对应这三个问题：localization、repair、patch validation。这种拆法成立，是因为 repo repair 通常有一个局部性假设：最终 patch 往往只改少量文件、少量函数、少量行。只要 localization 能保留真实编辑位置，repair 就不需要看完整仓库。

问题	Agentless 表达	失败代价
在哪里改	location set	真实位置掉出候选集
怎么改	patch samples	补丁错误或无法 apply
信哪个	tests + voting	错 patch 被选中

这三个问题之间有清晰因果关系。localization 的召回不足，后面再强的 repair 也很难成功；repair 候选太少，validation 没有可选对象；validation 信号弱，多个 plausible patches 之间就会选错。Agentless 的设计价值在于把这些失败点拆开，使得系统可以逐段调试。

3.1 把复杂任务拆成两个分布

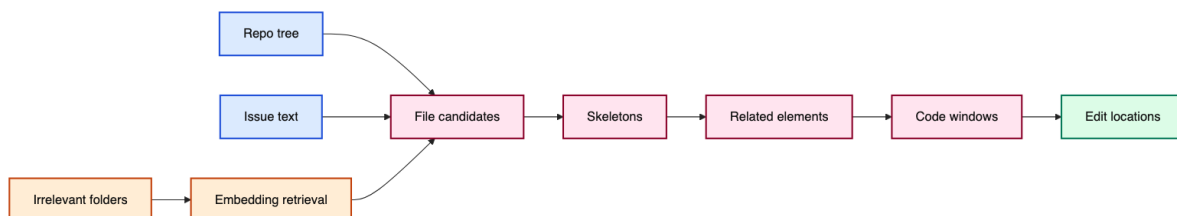
从机器学习的角度看，Agentless 不是在直接学习一个从 issue 到 final patch 的单步映射。它把问题拆成两个分布：一个是 candidate distribution，也就是尽可能覆盖正确位置和正确 patch；另一个是 selection distribution，也就是在候选集中把坏 patch 排到后面。

这个拆法降低了每次 LLM 调用的难度。模型做 localization 时不需要生成 patch，做 repair 时不需要决定全局探索路线，做 test selection 时不需要重新理解代码实现。每个阶段都只需要解决一个局部问题。但代价是 stage boundary 必须设计得足够好：如果上一阶段输出的信息不足，下一阶段没有办法靠“聪明”自动补回来。

Insight. Agentless 的系统美感在于，它承认 LLM 会犯错，所以不押注单次完美推理，而是用多候选和可执行信号把错误显式暴露出来。

4 Localization funnel

Agentless 的 localization 不是一次性让模型找行号，而是一个逐步收窄的漏斗。



第一层是 file-level localization。系统把 repository 变成类似 tree 的目录结构，和 issue description 一起给 LLM，让它选 top suspicious files。这个表示不包含完整源代码，所以成本低，也不容易被长文件干扰。

为了补充只看文件名的不足, Agentless 同时做 embedding-based retrieval。它先让 LLM 标出 irrelevant folders, 再在剩余文件中切 code chunks, 用 text-embedding-3-small 计算 embedding; 论文默认 chunk size 为 512, overlap 为 0。最后把 LLM-predicted files 和 embedding-retrieved files 合并, 得到最终 suspicious files。

4.1 三层定位

第一层: file candidates。 目标是不要漏掉可能相关文件。论文 ablation 中, prompting-based 和 embedding-based 方法互补; 组合后 file-level localization 的 Contains GT 达到 81.67%。这里的重点是召回, 而不是精确行号。

第二层: related elements。 对每个 suspicious file, Agentless 不直接给完整文件, 而是给 skeleton format: 类、函数、变量声明, 类字段、方法签名, 以及部分 module/class comments。模型用这个压缩结构选择相关 class/function/variable。

第三层: edit locations。 系统再把相关元素的完整代码给模型, 让它输出更细的编辑位置, 可以是类、函数或具体行。论文默认会对 edit locations 采样 4 组, 而不是只信一个 greedy 结果。

定位漏斗的目标不是一次命中,
而是在每一层尽量减少上下文, 同时尽量不把真实编辑位置筛掉。

5 Skeleton format 的作用

Skeleton format 是 Agentless 中最有教学价值的中间表示。它回答一个很实际的问题: 模型既不能看完整仓库, 也不应该只看文件名; 那么在 file 和 full code 之间, 应该给它什么?

表示	保留什么	丢掉什么
Repo tree	路径和文件组织	源码语义
Skeleton	类/函数/变量结构	函数体细节
Code window	局部完整代码	文件全局上下文
Full file	全部源代码	无压缩

Skeleton 的价值在于, 它保留了代码的组织结构。模型可以看到一个文件里有哪些类、哪些方法、哪些 top-level functions, 从而判断 issue 可能涉及哪个元素; 但它不会被完整函数体、长注释、无关 helper 代码淹没。

论文 ablation 显示, related-element localization 用 skeleton format 比直接用 complete file 更好, 且成本低得多。完整文件看似信息更多, 但会让模型在长上下文中分心; skeleton 反而把“选择哪里进一步细看”这个任务变得更清楚。

5.1 Skeleton 的失败模式

Skeleton 也不是万能的。它压缩掉了函数体, 因此看不到具体控制流、异常处理、边界条件和变量之间的数据依赖。如果 issue 的关键线索藏在函数体内部, 例如某个 helper 返回空列表、某个缓存状态只在特定分支更新, skeleton 可能只能把模型带到“大概相关”的函数, 不能保证精确行号。

所以 Agentless 不是只用 skeleton 修复问题, 而是把 skeleton 放在中间层。它的任务是把完整文件压缩成可选择的结构索引, 然后再把被选中的 related elements 展开成完整代码。这个顺序很关键: 先压缩再展开, 而不是一直压缩, 也不是一开始就塞完整文件。

Insight. Skeleton 是一种压缩后的可行动上下文。它不是为了让模型直接修 bug，而是为了让模型更好地决定下一层应该看哪里。

6 Repair as candidate generation

定位之后，Agentless 不让模型在环境里来回编辑，而是把修复变成候选 patch 生成。

对每个 edit location，系统构造一个 code context window。论文默认会在定位位置前后各加 10 行上下文。如果有多个 edit locations，就把这些窗口拼接起来，中间用省略标记表示缺失上下文。这样模型看到的是“足够修复”的局部代码，而不是整文件。

Patch format 也被约束。Agentless 使用 Search/Replace edit，而不是让模型生成完整文件。

Patch 表达	优点	风险
完整文件	输出直接但很长	幻觉、重复、覆盖无关代码
Line edit	精确但行号脆弱	行号漂移、格式错误
Search/Replace	小 diff、可 apply	search block 必须匹配

Search/Replace 的思想是：给出原始代码片段和替换代码片段。系统在文件中找到 search block，再替换成 replace block。它把模型的生成目标限制成小编辑，减少生成整文件时的 hallucination 和冗余。

6.1 Search/Replace 的隐含约束

Search/Replace 看起来只是输出格式，实际上是一种强约束。模型必须复制出足够准确的原始代码片段，系统才能找到匹配位置；替换代码也必须保持缩进、作用域和局部变量一致。这会过滤掉一部分“语义上想得对、格式上无法 apply”的回答。

这个约束有两面性。好处是 patch 更小、更容易审计，也更接近真实 code review 中的 diff。坏处是如果 search block 太短，可能匹配到错误位置；如果太长，轻微空格或上下文差异都会导致 apply 失败。因此 repair 阶段的上下文窗口和 edit location 质量会直接影响 patch 可应用性。

6.2 为什么要采样 40 个 patch

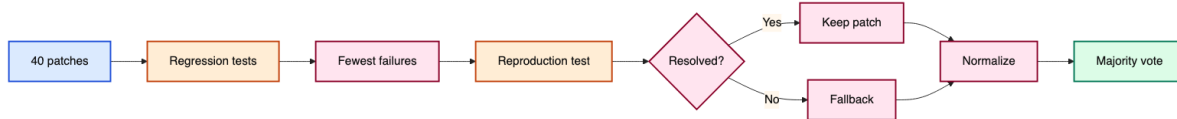
Agentless 默认采样 4 组 edit locations；每组生成 10 个 patches，包含 1 个 greedy sample 和 9 个 temperature samples，所以每个 issue 共有 40 个 patch candidates。这个数字不是为了炫耀采样量，而是因为 pipeline 后面有 validation，可以从候选集中挑选。

这和 autonomous agent 的单一路径不同。一个 agent trajectory 如果早期定位错了，后续 patch 大概率都跟着错；Agentless 则让不同 location sets 独立进入 repair。只要其中一组定位保留了真实 bug，patch sampling 就有机会生成正确修复。

Agentless 把 repair 从“走一条正确路线”变成“生成候选并筛选”。
这要求系统有足够强的 validation，否则采样只会制造更多 plausible but wrong patches。

7 Patch validation

候选 patch 多了之后，关键问题变成选择。Agentless 的 patch validation 用两类测试：regression tests 和 generated reproduction tests。



7.1 Regression tests

Regression tests 用来约束“不要破坏旧行为”。Agentless 先运行 repository 中已有测试，找出原始代码库上能通过的 tests。然后它让 LLM 根据 issue description 从这些 passing tests 中移除不应作为 regression 的测试，因为有些旧测试可能正好覆盖即将改变的行为。

论文特别说明，它没有直接使用 SWE-bench 提供的 PASS_TO_PASS 字段，而是从完整测试列表中选择可用 regression tests。这个选择让 validation 更接近“真实 issue 刚出现时，我们只能从现有代码库找旧行为约束”的场景。

7.2 Generated reproduction tests

Regression tests 只能证明旧行为没坏，不能证明当前 issue 修好了。Agentless 因此让 LLM 根据 issue description 生成 reproduction test。这个 test 被要求输出三类信号：Issue reproduced、Issue resolved、Other issues。

系统会采样 40 个 reproduction tests，并在原始 repository 上执行，过滤掉不能输出 Issue reproduced 的测试。然后对剩余测试做 normalize 和 majority selection，选出一个 reproduction test 用于 patch selection。

这里有一个重要限制：生成的 reproduction test 可能复现问题，却不能正确判断 patch 是否真正修好。论文报告，300 个 SWE-bench Lite 问题中，Agentless 能生成 213 个在原始 repo 上复现 issue 的测试；但应用 ground-truth patch 后，只有 94 个正确输出 resolved。这说明 generated tests 是有用信号，但不是可靠 oracle。

Insight. Agentless 对生成测试采取保守策略：reproduction test 可以帮助选择 patch，但不能凌驾于 regression tests 之上。测试生成越不可靠，selection 越需要 fallback。

7.3 Validation 不是 feedback loop

这里容易把 Agentless 和 RL 或 interactive debugging 混在一起。Agentless 的 validation 主要用于**选择**，不是用于让模型看到失败后继续修。也就是说，patch 失败通常不会触发一个新的长期交互轨迹，而是作为候选排序信号进入 reranking。

机制	失败后发生什么	系统含义
Interactive agent	模型观察报错并继续行动	online control
Agentless validation	标记候选 patch 的测试结果	offline selection
RL rollout	把结果变成训练信号	policy optimization

这个区别解释了为什么 Agentless 简洁。它不需要为每个失败 patch 构造新的 prompt trajectory，也不需要让模型解释测试日志再二次修复。复杂性被压在候选规模和 reranking 规则里，而不是压在线上行动 loop 里。

8 Normalization and majority voting

Agentless 的最后一步不是直接选第一个通过测试的 patch，而是排序、归一化、投票。

首先，系统对所有 candidate patches 运行 regression tests，保留 regression failures 最少的 patch。然后在这些 patch 中运行 selected reproduction test，只保留输出 Issue resolved 的 patch。如果没有 patch 通过 reproduction test，系统回退到只使用 regression result。

接着，系统对 patch 做 normalization。repo 的 rerank.py 会使用 normalized patch key，把空格、换行、注释等表面差异尽量消掉；论文还描述了基于 AST parse/unparse 的标准化流程。最后，系统对 normalized patches 做 majority voting，选择出现次数最多的候选作为最终提交。

选择信号	作用	不能保证什么
Regression pass	不破坏旧测试	修了当前 issue
Reproduction resolved	针对 issue 的修复信号	测试本身正确
Patch majority	多次采样一致	一致的 patch 必然正确
Normalization	合并表面差异	语义等价完全判定

这个选择过程体现了 Agentless 的系统哲学：没有单个信号足够可靠，所以把多个弱信号组合起来。Regression tests、reproduction tests、majority voting 各自都有盲点，但组合后比随机选择或只看模型置信度更稳。

8.1 为什么 majority voting 需要 normalization

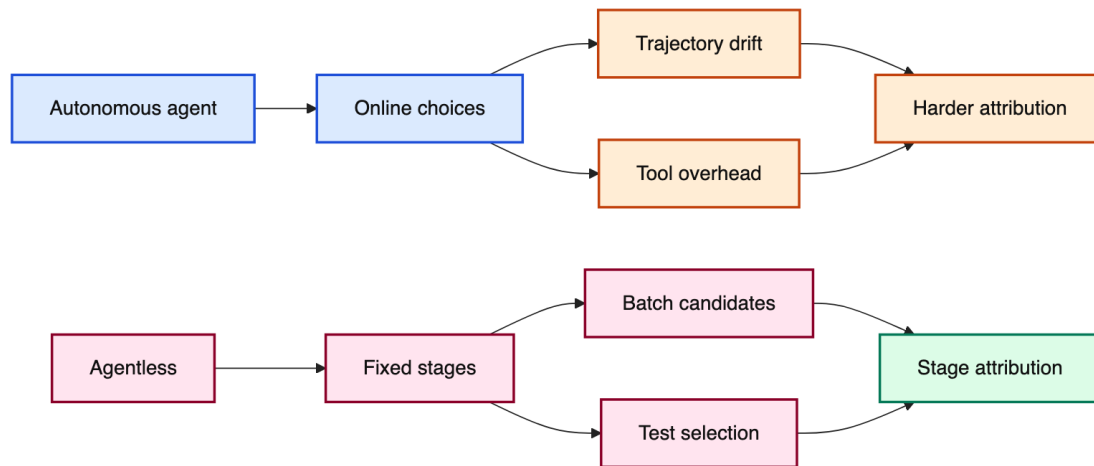
如果不做 normalization，两个本质相同的 patch 可能因为空格、注释、导入顺序或代码格式不同而被当成两个候选。这样投票会低估“同一个修复思路”的支持度。Normalization 的目标不是证明两个 patch 完全语义等价，而是把明显表面的差异合并掉，让 voting 更接近“修复策略投票”，而不是“字符串投票”。

这也说明 majority voting 的含义不能过度解读。一个 patch 被多次采样出来，说明它在模型分布里是高概率修复，不说明它一定正确。Agentless 之所以先用 regression 和 reproduction 过滤，再做 voting，就是为了避免“多数幻觉”直接获胜。

9 为什么少一点自治会帮忙

Agentless 的性能结果提供了一个反直觉信号：在论文报告的 SWE-bench Lite 设置中，Agentless with GPT-4o 解决 96/300 个问题，即 32.00%，平均成本约 \$0.70。这个结果不是全场最高，但在 open-source approaches 中很有竞争力，并且成本显著低于许多 agent-based methods。

更重要的是，这个结果来自一个非常朴素的系统。它没有复杂工具交互，没有持续环境观察，没有在线 planning，也没有多 agent 协作。它依赖的是分解、采样、验证和投票。



9.1 少自治带来的四个好处

第一，stage attribution 更清楚。 如果 final patch 失败，可以检查是 file localization 漏掉真实文件，还是 skeleton 阶段选错函数，还是 repair 没生成正确 patch，还是 validation 选错候选。

第二，候选可以并行。 4 组 edit locations、40 个 patches、40 个 reproduction tests 都是 batch artifacts，不必等待一个在线 agent trajectory 一步步演化。

第三，成本更可控。 每个阶段的 sample 数、top-N 文件数、context window、test 执行范围都是明确参数。相比自由 agent loop，它更容易预算、复现和调参。

第四，错误不会沿一条 trajectory 无限传播。 一个错误 location set 只影响它对应的 patch samples；其他 location sets 仍然可能成功。validation 阶段还有机会筛掉坏候选。

9.2 这不是说 agent loop 没用

Agentless 的优势来自任务结构。如果 issue text 包含足够线索，bug 的位置相对局部，patch 可以被 small diff 表达，测试可以执行，那么 pipeline 很强。反过来，如果任务需要安装依赖、探索动态行为、调试环境、打开浏览器、与用户澄清需求，固定 pipeline 就会显得僵硬。

所以正确结论不是“agentless 永远更好”，而是：**自治应该按任务需要购买。** 每增加一层自治，都应该问它是否真的带来信息增益，还是只是在增加工具开销和轨迹漂移。

10 Benchmark 信息质量

Agentless 论文有一个很重要但容易被忽略的部分：它不只报告分数，还分析 SWE-bench Lite 的任务质量。这个分析对我们理解 coding-agent benchmark 很关键，因为 repo issue repair 的难度不只来自代码，也来自 issue description 给了多少信息。

Issue 信息形态	对 Agentless 的影响	对 benchmark 的启发
有明确 location clue	localization 更容易保留真位置	分数部分来自题面线索
描述含 solution steps	patch generation 被显著简化	可能高估真实开发能力
描述信息不足	pipeline 难以补全缺失上下文	需要交互式搜索或人工澄清
misleading solution	validation 更容易选错 plausible patch	需要看最终可执行行为

这部分不是给 Agentless 找借口，而是在提醒我们：评价 long-horizon coding agent 时，不能只看 resolved rate。一个系统可能擅长从明确 issue 中做局部修复，却不擅长从模糊 bug report 中探索根因；另一个系统可能分数略低，但在无位置线索、需要动态调试的任务上更强。benchmark 的信息结构决定了我们测到的是哪一种能力。

Insight. SWE-bench 分数是结果，不是能力分解。要判断一个 coding agent 是否真正强，需要同时看 issue 信息、定位召回、patch 可执行性、测试信号和选择策略。

11 局限与开放问题

Agentless 的局限也很清楚。

Localization 仍然是瓶颈。

如果 issue description 没有足够 file/function clues，repo tree 和 embedding retrieval 都可能找不到真实位置。论文的分类分析也显示，完全没有 location clues 的问题更难，closed-source agent tools 在这类问题上可能更有优势，因为它们可以进行复杂交互式搜索。

Generated reproduction tests 不是 oracle。

213/300 的 generated tests 能在原始 repo 上复现 issue，但只有 94 个在 ground-truth patch 后正确输出 resolved。这说明 test generation 可以提供弱监督，但不能当作绝对真值。一个错误 reproduction test 可能奖励错误 patch。

Pipeline 对环境问题不敏感。

如果任务失败原因来自 dependency setup、flaky tests、隐藏服务、交互式 UI 或动态状态，Agentless 的固定阶段可能难以处理。Autonomous agent 可以在环境里探索并调整策略，而 pipeline 只能按预设流程推进。

Benchmark artifacts 会影响结论。

论文发现 SWE-bench Lite 中存在 issue description 信息不足、solution steps 甚至 exact patch 出现在描述中、misleading solution 等问题。Agentless 的 benchmark study 提醒我们：pipeline 的高分可能来自真实能力，也可能受到 benchmark 信息质量影响。评价这类系统时，必须同时看任务分布和中间轨迹。

12 系统设计启发

Agentless 给 coding-agent 系统设计带来三个启发。

1. 先判断任务能否被拆成候选生成和候选选择。如果可以，pipeline 往往比在线 agent 更容易调试。
2. 对于 repo repair，不要急着让模型自由探索。先构造强 localization funnel，再决定是否需要 agentic search。
3. Validation 应该成为系统一等公民。没有 regression/reproduction/normalization/voting，multi-sample repair 只会产生更多看似合理的错误 patch。

这也解释了为什么 Agentless 对后续系统有影响。许多产品或 agent 平台即使保留 autonomous loop，也会借鉴它的局部思想：先批量找候选位置，生成多个 patch，执行 tests，再由 ranking 或 reviewer 选择。

13 最终 takeaway

Agentless 可以压缩成三句话。

- 它不是不用 LLM，而是不让 LLM 自由控制整个行动 loop。
- 它把 repo repair 变成 localization、repair、validation 三段搜索与选择问题。
- 当任务有可定位上下文和可执行验证时，少一点自治可以减少噪声、降低成本，并提高可审计性。

因此，读 Agentless 最重要的不是记住 32.00% 的分数，而是理解一个更一般的系统原则：**agentic autonomy 是设计变量，不是默认目标**。对某些长期编码任务，最强系统未必是最自由的系统，而是把模型能力放进正确约束、正确候选集和正确验证信号里的系统。

参考资料

- [1] [Agentless paper](#). deterministic localization-repair-validation pipeline and SWE-bench analysis
- [2] [Agentless repository](#). implementation of localization, repair, reproduction tests, and patch reranking
- [3] [SWE-bench](#). repository issue repair benchmark used for Agentless evaluation